

ARM Cortex-M3 与 Cortex-M4

处理器指令集详解

—— 第5章 学习笔记 ——

基于《ARM Cortex-M3与Cortex-M4权威指南（第3版）》

原著：Joseph Yiu

ARM 嵌入式系统经典教材

涵盖章节：5.1 ~ 5.9 | 指令集详解 | Cortex-M4 DSP扩展 | 浮点指令

目录

附：学习总结与重点回顾

5.1 ARM Cortex-M 指令集背景简介

5.2 Cortex-M 处理器指令集比较

5.3 汇编语言语法

5.4 指令后缀

5.5 统一汇编语言 (UAL)

5.6 指令集详解

5.6.1 数据传送

5.6.2 存储器访问（9种寻址模式）

5.6.3 算术运算

5.6.4 逻辑运算

5.6.5 移位和循环移位

5.6.6 数据转换

5.6.7 位域处理

5.6.8 比较和测试

5.6.9 程序流控制

5.6.10 饱和运算

5.6.11 异常相关指令

5.6.12 休眠模式指令

5.6.13 存储器屏障

5.6.14 其他指令

5.6.15 不支持的指令

5.7 Cortex-M4 特有指令（DSP/SIMD/FPU）

5.8 桶形移位器

5.9 访问特殊寄存器和特殊指令

5.1 ARM Cortex-M 指令集背景简介

ARM 指令集架构（ISA）经历了数十年的演进，从最初的32位ARM指令集逐步发展到今天的Thumb-2技术。理解其历史脉络，对于深入掌握Cortex-M处理器的指令集至关重要。

ARM ISA 演进时间线

ARM版本	代表处理器	关键特性	应用定位
ARMv4	ARM7 (ARM7TDMI)	32位ARM指令 + 16位Thumb指令	1990年代后期
ARMv4T	ARM7TDMI, ARM9TDMI	引入Thumb 16位指令集，代码密度大幅提升	经典嵌入式
ARMv5TE	ARM9E, ARM10E	增强DSP指令（饱和运算、MAC）	信号处理
ARMv6-M	Cortex-M0, M0+, M1	仅Thumb-1子集（56条指令），极简设计	低成本MCU
ARMv7-M	Cortex-M3	Thumb-2技术（16+32位），硬件除法、位域操作	高性能嵌入式
ARMv7E-M	Cortex-M4, M7	DSP扩展、SIMD、可选单精度FPU	数字信号控制
ARMv8-M	Cortex-M23, M33	TrustZone安全扩展	IoT安全应用

核心概念

- Thumb技术：16位指令集，相比32位ARM指令可将代码尺寸缩小约30%，同时保持大部分性能。特别适合存储器带宽受限的嵌入式系统。
- Thumb-2技术：混合使用16位（高代码密度）和32位指令（完整功能），无需处理器状态切换，是Cortex-M3/M4的核心技术。传统ARM处理器需要BX指令在ARM状态和Thumb状态间切换，Thumb-2彻底消除了这一复杂性。
- Cortex-M仅支持Thumb状态：与经典ARM处理器（ARM7/ARM9）不同，Cortex-M系列处理器仅支持Thumb/Thumb-2指令，不支持32位ARM指令。这意味着所有指令都以Thumb模式执行。

关键点

注意：注意：Cortex-M处理器不能直接运行ARM7TDMI的ARM（32位）二进制代码，因为Cortex-M不支持ARM指令集。但是，将ARM7TDMI的Thumb代码移植到Cortex-M是简单的，且可以进一步利用Thumb-2指令优化性能。

Cortex-M3引入的Thumb-2指令集包含约130条指令，覆盖了几乎所有传统ARM指令集的功能，同时保持了Thumb的高代码密度优势。Cortex-M4在此基础上进一步添加了DSP和浮点扩展。

5.2 Cortex-M 处理器指令集比较

ARM Cortex-M系列处理器覆盖了从简单微控制器到高性能数字信号处理器的广泛需求。下表总结了各型号指令集的差异。

处理器	架构版本	指令集	特色整数指令	浮点/DSP	典型应用
Cortex-M0/M0+	ARMv6-M	基本Thumb-1 (56条)	无	无	低成本控制
Cortex-M1	ARMv6-M	基本Thumb-1	无	无	FPGA软核
Cortex-M3	ARMv7-M	Thumb-2完整	硬件除法(UDIV/SDIV) 位域操作(BFC/BFI/UBFX等) MAC指令	无	高性能嵌入式控制
Cortex-M4	ARMv7E-M	Thumb-2 + DSP扩展	同M3 + SIMD 饱和运算增强	SIMD并行处理 DSP指令 可选单精度FPU	数字信号控制 音频/传感器
Cortex-M7	ARMv7E-M	Thumb-2 + DSP + FP	同M4 + 双精度可选	可选双精度FPU 六级超标量流水线	高端DSP+控制

向上兼容性

- M0/M0+ → M3/M4: Cortex-M0/M0+的二进制代码可直接在Cortex-M3和M4上运行，因为M0的指令是Thumb-1的子集，M3/M4完全支持Thumb-2（包含Thumb-1）。
- M3 → M4: Cortex-M3的代码可在M4上运行。M4是M3的超集，保留了M3的所有指令并添加了DSP/SIMD扩展。
- 并非向下兼容: 使用了M4特有指令（SIMD/DSP/FPU）的代码不能在M3上运行，使用了Thumb-2 32位指令的代码也不能在M0上运行。

选型指南

- 基本I/O控制、简单传感器 → Cortex-M0/M0+（低成本、低功耗）
- 复杂数据处理、通信协议栈 → Cortex-M3（Thumb-2完整支持、硬件除法）
- 数字信号处理、音频、浮点运算 → Cortex-M4（DSP/SIMD/FPU）
- 高性能实时DSP+控制 → Cortex-M7（双精度、六级流水线）

提示： CMSIS-DSP库提供了大量优化的DSP函数（FFT、FIR、矩阵运算等），可在所有Cortex-M处理器上使用，但在M4/M7上利用SIMD和FPU可获得更高性能。

5.3 汇编语言语法

Cortex-M编程中最常使用的是ARM汇编器语法（ARM Compiler/Keil MDK）和GNU汇编器语法（GCC）。两者在指令格式、注释符号和伪指令方面存在差异，但核心指令助记符基本一致。

5.3.1 ARM 汇编器语法

```
label mnemonic operand1, operand2, ... ; 注释

MOVS R0, #0x55 ; 将立即数0x55加载到R0
ADD R1, R2, R3 ; R1 = R2 + R3
LDR R0, =0x20000000 ; 伪指令：加载32位常量
```

5.3.2 GNU 汇编器语法 (GAS)

```
label: mnemonic operand1, operand2, ... @ 注释 (也可用 /* */)

MOVS R0, #0x55 @ 将立即数0x55加载到R0
ADD R1, R2, R3 @ R1 = R2 + R3
LDR R0, =0x20000000 @ 伪指令：加载32位常量
```

5.3.3 常用伪指令对照表

ARM汇编器	GNU汇编器	功能说明	示例
DCB	.byte	定义字节 (8位) 常量	DCB 0x12, 0x34
DCW / DCWU	.hword / .2byte	定义半字 (16位) 常量	DCW 0x1234
DCD / DCDU	.word / .4byte	定义字 (32位) 常量	DCD 0x12345678
DCQ	.quad / .8byte	定义双字 (64位) 常量	DCQ 0x123456789ABCDEF0
DCFS	.float	定义单精度浮点数	DCFS 3.14159
DCFD	.double	定义双精度浮点数	DCFD 3.14159265
EQU	.equ / .set	定义符号常量	MY_CONST EQU 100
ALIGN	.align / .balign	地址对齐	ALIGN 4 (4字节对齐)
EXPORT / GLOBAL	.global	导出符号 (全局可见)	EXPORT myFunc
IMPORT / EXTERN	.extern	导入外部符号声明	IMPORT extFunc
AREA / SECTION	.section / .text / .data	定义代码/数据段	AREA MY_DATA, DATA
END	.end	汇编文件结束	END
SPACE / FILL	.space / .skip	预留内存空间	SPACE 256 (预留256字节)
LTORG	.ltorg	放置文字池 (literal pool)	LTORG

5.3.4 操作数约定

- 数据处理指令：第一个操作数通常是目标寄存器（如 ADD R0, R1, R2 中 R0 是目标）。

- 加载指令 (Load): 第一个操作数是被加载的寄存器 (如 LDR R0, [R1] 中 R0 从内存加载)。
- 存储指令 (Store): 第一个操作数是数据源寄存器 (如 STR R0, [R1] 中 R0 存储到内存)。
- 立即数前缀: ARM汇编器用 # 表示立即数 (如 #0x55), GNU汇编器同样使用 # 前缀。
- 寄存器范围: R0-R12为通用寄存器, R13=SP (堆栈指针), R14=LR (链接寄存器), R15=PC (程序计数器)。

5.4 指令后缀

Thumb-2指令集通过后缀（suffix）来精确控制指令行为。后缀可以组合使用，是实现灵活编程的关键机制。

5.4.1 S 后缀 — 更新APSR标志

在UAL（统一汇编语言）中，默认情况下数据处理指令不更新APSR标志位。如果需要更新N/Z/C/V/Q标志，必须在指令助记符后添加 S 后缀。

```
ADDS  R0, R1, R2 ; R0 = R1 + R2, 更新 N, Z, C, V
ADD   R0, R1, R2 ; R0 = R1 + R2, 不更新标志位
SUBS  R0, R0, #1  ; 递减并更新标志（常用于循环计数）
MOVS  R0, #0      ; 传送并更新标志
```

注意：这是UAL相比传统Thumb最重要的变化之一。在传统Thumb中，16位数据处理指令自动更新APSR；在UAL中，必须显式使用S后缀。

5.4.2 条件码后缀

条件码用于条件分支（B<cond>）和IT指令块中。14个条件码覆盖了各种比较结果：

后缀	编码	含义	标志条件	用法示例
EQ	0000	Equal (相等)	Z == 1	BEQ label
NE	0001	Not Equal (不相等)	Z == 0	BNE label
CS / HS	0010	Carry Set / Unsigned Higher or Same	C == 1	BCS label
CC / LO	0011	Carry Clear / Unsigned Lower	C == 0	BCC label
MI	0100	Minus / Negative (负数)	N == 1	BMI label
PL	0101	Plus / Positive or Zero (正数/零)	N == 0	BPL label
VS	0110	Overflow Set (溢出置位)	V == 1	BVS label
VC	0111	Overflow Clear (溢出清除)	V == 0	BVC label
HI	1000	Unsigned Higher (无符号大于)	C==1 AND Z==0	BHI label
LS	1001	Unsigned Lower or Same (无符号小于等于)	C==0 OR Z==1	BLS label
GE	1010	Signed Greater or Equal (有符号大于等于)	N == V	BGE label
LT	1011	Signed Less Than (有符号小于)	N != V	BLT label
GT	1100	Signed Greater Than (有符号大于)	Z==0 AND N==V	BGT label
LE	1101	Signed Less or Equal (有符号小于等于)	Z==1 OR N!=V	BLE label

5.4.3 .N 和 .W 后缀

- .N (Narrow): 强制汇编器生成16位（窄）指令编码。如果指令不支持16位形式，汇编器报错。
- .W (Wide): 强制汇编器生成32位（宽）指令编码。默认情况下汇编器会自动选择最紧凑的编码，使用.W可覆盖自动选择。

```
ADD.N R0, R1, R2 ;强制16位编码（仅低寄存器可用）
ADD.W R0, R1, R2 ;强制32位编码（支持高寄存器）
B.W target ;强制32位分支（可达±16MB范围）
```

5.4.4 浮点后缀 .32/.F32 和 .64/.F64

- .32 或 .F32: 单精度（32位）浮点操作，用于FPv4-SP FPU（Cortex-M4可选）。
- .64 或 .F64: 双精度（64位）浮点操作，仅在Cortex-M7等支持双精度FPU的型号上可用。

```
VADD.F32 S0, S1, S2 ;单精度浮点加法 S0 = S1 + S2
VMUL.F64 D0, D1, D2 ;双精度浮点乘法 D0 = D1 * D2
```

5.4.5 APSR 标志位详解

标志位	名称	描述	说明
N (bit 31)	Negative (负标志)	运算结果的最高位 (bit 31)	N=1 表示结果为负数
Z (bit 30)	Zero (零标志)	运算结果全零时为1	Z=1 表示结果为零
C (bit 29)	Carry/Borrow (进位/借位)	加法进位或减法借位；移位溢出的最后一位	ADD产生进位时C=1
V (bit 28)	oVerflow (溢出标志)	有符号运算溢出	正+正=负，或负+负=正时V=1
Q (bit 27)	Saturation (饱和标志)	饱和运算发生饱和时置位	写1后保持为1，需手动清除

5.5 统一汇编语言 (UAL)

统一汇编语言 (Unified Assembler Language, UAL) 是ARM为Thumb-2架构定义的标准汇编语法，解决了传统Thumb和ARM指令集语法不统一的问题。对于从旧代码迁移的开发者，理解UAL与传统Thumb的差异至关重要。

UAL三大关键变化

变化一：S后缀显式化

在传统Thumb汇编中，16位数据处理指令总是自动更新APSR标志位。例如 `ADD R0, R1, R2` 会自动更新N/Z/C/V。在UAL中，必须显式使用 `S` 后缀才能更新标志位 (`ADDS R0, R1, R2`)。这提供了更精确的控制。

警告：如果在UAL中使用旧的Thumb代码（没有S后缀的数据处理指令），汇编器不会自动添加S后缀，导致标志位不被更新。依赖标志位的后续条件指令（如条件分支）可能产生错误结果！

变化二：三操作数格式推荐

传统Thumb中许多指令只能使用两操作数格式（目标也是源操作数之一）。UAL推荐使用三操作数格式：`ADD R0, R1, R2` ($R0 = R1 + R2$)，而非两操作数格式 `ADD R0, R1` ($R0 = R0 + R1$)。三操作数格式更清晰，且能更好地被汇编器优化。

```
; UAL 三操作数 (推荐)
ADD    R0, R1, R2    ; R0 = R1 + R2, 源和目标分离
SUB     R3, R4, R5    ; R3 = R4 - R5

; 传统两操作数 (仍支持, 但不推荐)
ADD     R0, R1        ; R0 = R0 + R1 目标=源1
```

变化三：.N和.W宽度后缀

UAL引入了 `.N` (Narrow, 16位) 和 `.W` (Wide, 32位) 后缀来控制指令编码宽度。不使用后缀时，汇编器自动选择最紧凑的合法编码。在与旧代码混合时，使用 `.N` 和 `.W` 可以精确控制指令长度——这在需要确保指令对齐或特定时序的场景中非常有用。

其他重要说明

- 32位Thumb-2指令可以半字对齐：与ARM指令（必须4字节对齐）不同，32位Thumb-2指令只需半字对齐（2字节），这进一步提升了代码密度。处理器取指单元会自动处理非4字节对齐的32位指令。
- 立即数表示：UAL中立即数可用 `#` 前缀（如 `#0x55`），或直接用数字。GNU汇编器的 `.syntax unified` 指令可使GAS使用UAL语法。

```
; GNU汇编器中启用UAL语法
.syntax unified
.thumb
ADDS    R0, #1        ; UAL格式
```

5.6 指令集详解

本节逐一详解Cortex-M3/M4处理器的所有指令类别。每个子节包含指令表、用法说明和实例代码。这是本章最核心和最长的一个小节，涵盖了15个子类别。

5.6.1 数据传送指令

数据传送指令用于在寄存器之间、寄存器和特殊寄存器之间以及寄存器和立即数之间移动数据。

指令格式	功能	示例
MOV Rd, <op2>	将操作数传送至Rd	MOV R0, R1 MOV R0, #100
MOVS Rd, <op2>	传送并更新APSR标志	MOVS R0, #0 ; Z=1
MOVW Rd, #imm16	将16位立即数写入Rd低16位（高16位清零）	MOVW R0, #0x1234
MOVT Rd, #imm16	将16位立即数写入Rd高16位（低16位不变）	MOVT R0, #0x5678 ; R0=0x56781234
MVN Rd, <op2>	按位取反后传送	MVN R0, #0 ; R0=0xFFFFFFFF
MRS Rd, <spec_reg>	读特殊寄存器到通用寄存器	MRS R0, PRIMASK
MSR <spec_reg>, Rn	写通用寄存器到特殊寄存器	MSR CONTROL, R0

FPU 数据传送指令

指令格式	功能	示例
VMOV.F32 Sd, Sm	单精度浮点寄存器间传送	VMOV.F32 S0, S1
VMOV Rt, Sn	ARM寄存器 → 浮点寄存器（传送位模式）	VMOV R0, S0
VMOV Sn, Rt	浮点寄存器 → ARM寄存器	VMOV S0, R0
VMRS Rt, FPSCR	读浮点状态控制寄存器	VMRS R0, FPSCR
VMSR FPSCR, Rt	写浮点状态控制寄存器	VMSR FPSCR, R0

加载32位立即数的方法

- 方法一：LDR伪指令 — 汇编器自动将32位立即数放入文字池（literal pool），然后生成一条PC相对LDR指令。使用简单，但需要注意文字池不能太远（通常4KB范围内）。
- 方法二：MOVW + MOVT — 分两次写入16位立即数：MOVW写低16位，MOVT写高16位。不依赖文字池，代码位置无关，推荐用于位置无关代码（PIC）。

```
; 方法一：LDR伪指令（依赖文字池）
LDR    R0, =0x12345678    ; 汇编器自动处理

; 方法二：MOVW + MOVT（推荐用于PIC）
MOVW   R0, #0x5678        ; R0低16位 = 0x5678
MOVT   R0, #0x1234        ; R0高16位 = 0x1234 → R0 = 0x12345678
```

文字池 (Literal Pool) 与 LTORG

文字池是汇编器在代码段中自动放置的32位常量数据块。当使用 `LDR Rd, =constant` 伪指令时，汇编器将常量放入文字池，并生成一条PC相对加载指令。LTORG 伪指令可显式控制文字池的放置位置，避免文字池插入到不可达范围或条件执行区域。

```
; 文字池控制
LDR    R0, =0x12345678    ; 引用文字池
...
; 代码
B      next               ; 跳过分隔
LTORG                      ; 在此放置文字池
next:
...
```

5.6.2 存储器访问指令（9种寻址模式）

存储器访问是处理器最核心的操作之一。Cortex-M3/M4支持丰富的数据传输指令和9种寻址模式，为不同场景提供最优的内存访问方式。

数据传输指令总览

指令	数据大小	说明	示例
LDRB / STRB	字节 (8位)	零扩展至32位	LDRB R0, [R1]
LDRSB	有符号字节	符号扩展至32位	LDRSB R0, [R1]
LDRH / STRH	半字 (16位)	零扩展至32位	LDRH R0, [R1]
LDRSH	有符号半字	符号扩展至32位	LDRSH R0, [R1]
LDR / STR	字 (32位)	完整32位	LDR R0, [R1]
LDM / STM	多寄存器	加载/存储多个寄存器	LDMIA R0!, {R1-R4}
LDRD / STRD	双字 (64位)	加载/存储两个32位寄存器	LDRD R0, R1, [R2]
PUSH / POP	堆栈操作	压入/弹出寄存器列表	PUSH {R4-R6, LR}

寻址模式详解

模式1：立即数偏移（Pre-indexed，前变址）

基址寄存器 + 立即数偏移量，计算出的地址用于访存。可选地，可以用！
后缀将计算结果写回基址寄存器（write-back），实现自动递增/递减，非常适合数组遍历。

```
; 立即数偏移（无写回）
LDR  R0, [R1, #8] ; R0 = [R1 + 8], R1 不变
STR  R0, [R1, #4] ; [R1 + 4] = R0, R1 不变

; 立即数偏移（带写回）
LDR  R0, [R1, #8]! ; R0 = [R1 + 8], 然后 R1 = R1 + 8
STR  R0, [R1, #4]! ; [R1 + 4] = R0, 然后 R1 = R1 + 4
```

提示：带！写回的寻址模式相当于C语言中的 *p++ 操作，对数组遍历和结构体成员访问非常高效。

模式2：PC相对寻址

以PC为基址，加上一个立即数偏移量。这是访问文字池（literal pool）中常量的标准方式。实际编程中常通过 LDR Rd, =constant 伪指令间接使用此模式。

```
; PC相对寻址（访问文字池中的常量）
LDR  R0, [PC, #100] ; R0 = [PC + 100], 仅向前
LDR  R0, =0x12345678 ; 汇编器自动转换为PC相对LDR
```

模式3：寄存器偏移（Pre-indexed with Register Offset）

有效地址 = 基址寄存器 +
偏移寄存器（可选左移0-3位）。移位量用于缩放索引，便于访问不同大小的数组元素。

```
; 寄存器偏移
```

```
LDR    R0, [R1, R2]      ; R0 = [R1 + R2]
LDR    R0, [R1, R2, LSL #2] ; R0 = [R1 + R2*4] 适合访问32位数组
```

模式4：后变址 (Post-indexed)

先用基址寄存器的原始值访问内存，然后自动更新基址寄存器（加上偏移量）。这种模式非常适合顺序处理数组元素——先使用当前值，再自动递增指针。

```
; 后变址寻址（自动更新基址）
LDR    R0, [R1], #4    ; R0 = [R1], 然后 R1 = R1 + 4
STR    R0, [R1], #4    ; [R1] = R0, 然后 R1 = R1 + 4

; 后变址数组处理示例
MOV    R0, #10         ; 循环10次
loop:
LDR    R2, [R1], #4    ; R2 = *R1, 然后 R1 += 4
; ... 处理 R2 ...
SUBS   R0, R0, #1
BNE    loop
```

模式5：多寄存器加载/存储 LDM/STM

一条指令加载或存储多个寄存器。常用变体：IA (Increment After, 后递增) 和 DB (Decrement Before, 前递减)，分别对应空堆栈和满堆栈操作。

```
; LDMIA / STMIA — 传后递增
LDMIA  R0!, {R1-R4}    ; 从[R0]连续加载4个字到R1-R4, R0递增16
STMIA  R0!, {R1-R4}    ; 将R1-R4连续存储到[R0], R0递增16

; LDMDB / STMDB — 传前递减
LDMDB  R0!, {R1-R4}    ; R0先减16, 再从[R0]连续加载
STMDB  R0!, {R1-R4}    ; R0先减16, 再将R1-R4连续存储

; 寄存器列表可以是非连续的
LDMIA  R0!, {R1, R3, R5-R7} ; 按寄存器编号升序传输
```

模式6：PUSH/POP（堆栈操作）

PUSH/POP 是 STMDB/STMIA 的别名（以SP为基址），专门用于堆栈操作。16位版本的PUSH/POP仅支持低寄存器（R0-R7）和LR/PC，32位版本支持所有寄存器。

```
; 函数入口 — 保存现场
PUSH  {R4-R7, LR}  ; 将R4-R7和LR压入堆栈

; 函数体 ...

; 函数出口 — 恢复现场并返回
POP   {R4-R7, PC}  ; 恢复R4-R7, PC=返回地址（等效于BX LR）
```

模式7：SP相对寻址

以堆栈指针（SP）为基址的寻址，常用于访问局部变量和函数参数。汇编器通常自动处理。

```
; SP相对寻址
LDR   R0, [SP, #8]  ; 加载SP+8处的局部变量
STR   R1, [SP, #4]  ; 存储到SP+4处的局部变量
```

模式8：非特权访问

带T后缀的指令（LDRT, STRT, LDRBT, STRBT, LDRHT, STRHT, LDRSBT, LDRSHT）即使在特权模式下也执行非特权访问。主要用于操作系统在特权模式下模拟用户模式的访存操作。

```
; 非特权访问（处理器在特权模式）
LDRT  R0, [R1]      ; 以用户权限从[R1]加载
```

模式9：独占访问（Exclusive Access）

LDREX/STREX/CLREX

用于实现信号量（semaphore）和互斥锁（mutex），是RTOS和多任务系统的关键指令。STREX返回0表示写入成功（获取锁成功），返回1表示失败（被其他任务抢先）。

```
; — 互斥锁 (Mutex) 实现 —
; 获取锁
MOV   R1, #1      ; 锁的"锁定"值
try_lock:
LDREX R0, [R2]     ; 独占读取锁变量
CMP   R0, #0       ; 锁是否空闲？
BNE   lock_busy    ; 非零 = 已被占用
STREX R0, R1, [R2]  ; 尝试独占写入1（加锁）
CMP   R0, #0       ; STREX 成功？
BNE   try_lock     ; 失败=被抢占，重试
; 获取锁成功
lock_busy:
...
; 释放锁
MOV   R0, #0
```

```
STR    R0, [R2]    ; 写入0释放锁
```

注意：STREX 可能失败的情况：(1) 总线级互斥访问失败（多主控总线冲突）；

(2) 本地独占监视器被清除——包括中断进入/退出、CLREX指令执行、其他地址的LDREX都会清除监视器。因此ISR中不应使用STREX。

5.6.3 算术运算指令

Cortex-M3/M4支持加法、减法、乘法和除法四类基本算术运算。M4的DSP扩展额外提供了SIMD和饱和运算变体（见5.7节）。算术指令中，仅带S后缀的变体会更新APSR标志。

指令格式	功能说明	示例
ADD Rd, Rn, <op2>	加法	ADD R0, R1, R2 ADD R0, R1, #5
ADC Rd, Rn, <op2>	带进位加法 (Rd=Rn+op2+C)	ADCS R0, R1, R2 ; 64位加法
ADDW Rd, Rn, #imm12	宽加法 (32位编码, 立即数12位)	ADDW R0, R1, #0x7FF
SUB Rd, Rn, <op2>	减法	SUB R0, R1, R2
SBC Rd, Rn, <op2>	带借位减法 (Rd=Rn-op2-NOT(C))	SBCS R0, R1, R2 ; 64位减法
SUBW Rd, Rn, #imm12	宽减法 (32位编码, 立即数12位)	SUBW R0, R1, #0x7FF
RSB Rd, Rn, <op2>	反向减法 (Rd=op2-Rn)	RSB R0, R1, #0 ; R0 = -R1
MUL Rd, Rn, Rm	32位乘法 (Rd=Rn*Rm[31:0])	MUL R0, R1, R2
MLA Rd, Rn, Rm, Ra	乘加 (Rd=Ra+Rn*Rm)	MLA R0, R1, R2, R3
MLS Rd, Rn, Rm, Ra	乘减 (Rd=Ra-Rn*Rm)	MLS R0, R1, R2, R3
SMULL RdLo,RdHi,Rn,Rm	有符号64位乘 (RdHi:RdLo=Rn*Rm)	SMULL R0, R1, R2, R3
SMLAL RdLo,RdHi,Rn,Rm	有符号64位乘加 (RdHi:RdLo+=Rn*Rm)	SMLAL R0, R1, R2, R3
UMULL RdLo,RdHi,Rn,Rm	无符号64位乘 (RdHi:RdLo=Rn*Rm)	UMULL R0, R1, R2, R3
UMLAL RdLo,RdHi,Rn,Rm	无符号64位乘加 (RdHi:RdLo+=Rn*Rm)	UMLAL R0, R1, R2, R3
UDIV Rd, Rn, Rm	无符号除法 (Rd=Rn/Rm)	UDIV R0, R1, R2
SDIV Rd, Rn, Rm	有符号除法 (Rd=Rn/Rm)	SDIV R0, R1, R2

重要说明

- MAC指令不自动更新APSR：MUL/MLA/MLS等乘法指令即使加S后缀也不更新APSR。这是ARMv7-M的明确规定——乘法结果的标志位需要通过CMP等指令显式测试。
- UDIV/SDIV 除零行为：当除数为0时，结果默认为0（不产生异常）。Cortex-M3/M4的NVIC中有一个可配置的除法异常（DIVBYZERO），可通过SCB->CCR的DIV_0_TRP位（bit 4）使能除零异常。
- 64位乘法的寄存器对：RdLo和RdHi必须是不相同的寄存器，且不能与Rn或Rm相同。
- 带进位加法/减法：ADC/SBC通常与ADDS/SUBS配对使用，实现64位或更大整数的运算。

```
; 64位加法示例: R1:R0 += R3:R2
ADDS  R0, R0, R2    ; 低32位相加, 更新C
ADCS  R1, R1, R3    ; 高32位 + 进位

; 64位减法示例: R1:R0 -= R3:R2
SUBS  R0, R0, R2    ; 低32位相减, 更新C (借位=NOT(C))
SBCS  R1, R1, R3    ; 高32位 - 借位
```


5.6.4 逻辑运算指令

逻辑运算指令提供按位的与、或、异或、位清除等操作，常用于位掩码处理、标志位操作和寄存器字段配置。

指令格式	功能说明	示例
AND Rd, Rn, <op2>	按位与	AND R0, R1, #0xFF ; 取R1低8位
ORR Rd, Rn, <op2>	按位或	ORR R0, R1, #0x80 ; 设置bit 7
BIC Rd, Rn, <op2>	位清除 (Rd=Rn AND NOT op2)	BIC R0, R1, #0x80 ; 清除bit 7
ORN Rd, Rn, <op2>	按位或非 (Rd=Rn OR NOT op2)	ORN R0, R1, R2
EOR Rd, Rn, <op2>	按位异或	EOR R0, R1, R2 ; 位翻转

注意：16位版本的逻辑运算指令（AND/ORR/BIC/EOR）只能使用低寄存器（R0-R7），且目标必须与第一源操作数相同。32位版本（.W）无此限制。ORN指令没有16位版本。

```
; 位操作的常见模式
AND    R0, R1, #0xFF ; 屏蔽高24位, 只保留低字节
ORR     R0, R0, #(1<<3) ; 设置bit 3
BIC     R0, R0, #(1<<5) ; 清除bit 5
EOR     R0, R0, #(1<<2) ; 翻转bit 2 (toggle)
ORN     R0, R0, R1 ; R0 = R0 | (~R1)
```

5.6.5 移位和循环移位指令

移位指令提供算术右移、逻辑左移/右移、循环右移和带进位循环右移。移位操作也可以通过桶形移位器嵌入到数据处理指令中（见5.8节），实现「移位+运算」的单指令组合。

指令格式	功能说明	示例
ASR Rd, Rm, <n> ASR Rd, Rn, Rs	算术右移（保持符号位）	ASR R0, R1, #4 ASR R0, R1, R2
LSL Rd, Rm, <n> LSL Rd, Rn, Rs	逻辑左移（低位补0）	LSL R0, R1, #2 LSL R0, R1, R2
LSR Rd, Rm, <n> LSR Rd, Rn, Rs	逻辑右移（高位补0）	LSR R0, R1, #3 LSR R0, R1, R2
ROR Rd, Rm, <n> ROR Rd, Rn, Rs	循环右移	ROR R0, R1, #8 ROR R0, R1, R2
RRX Rd, Rm	带进位循环右移（1位，C参与）	RRX R0, R1

为什么没有专门的"循环左移"指令？

ARM指令集中没有ROL（循环左移）指令。原因在于：循环左移N位 = 循环右移(32-N)位，两者结果完全一致，执行时间相同。单独提供循环左移只会增加电路复杂度而无性能收益。例如，ROR R0, R1, #24等效于循环左移8位。

```
; 移位操作示例
LSL  R0, R1, #2    ; R0 = R1 << 2 (乘以4)
LSR  R0, R1, #3    ; R0 = R1 >> 3 (无符号除以8)
ASR  R0, R1, #2    ; R0 = R1 >> 2 (有符号除以4, 保持符号)
ROR  R0, R0, #8    ; 字节交换（循环右移8位）
RRX  R0, R1        ; {R0, C} = {C, R1[31:1], R1[0]} 33位移位
```

5.6.6 数据转换指令

数据转换指令用于在不同数据宽度之间进行符号扩展和字节序转换。这些操作在处理来自外设、传感器和网络的数据时非常常见。

指令格式	功能说明	示例
SXTB Rd, Rm {,ROR #n}	有符号字节扩展 (bit[7]→bit[31:8])	SXTB R0, R1
SXTH Rd, Rm {,ROR #n}	有符号半字扩展 (bit[15]→bit[31:16])	SXTH R0, R1
UXTB Rd, Rm {,ROR #n}	无符号字节扩展 (高位补0)	UXTB R0, R1
UXTH Rd, Rm {,ROR #n}	无符号半字扩展 (高位补0)	UXTH R0, R1
REV Rd, Rm	字节反转 [31:24]↔[7:0], [23:16]↔[15:8]	REV R0, R1
REV16 Rd, Rm	半字内字节反转 (每16位内反转)	REV16 R0, R1
REVSH Rd, Rm	半字内字节反转 + 有符号扩展	REVSH R0, R1

实际操作示例

```
; 符号扩展示例
; R1 = 0x0000FF80 (bit[7]=1, 解释为有符号字节=-128)
SXTB  R0, R1      ; R0 = 0xFFFFF80 (符号扩展到32位)
UXTB  R0, R1      ; R0 = 0x00000080 (无符号扩展=128)

; R1 = 0x00008000 (bit[15]=1)
SXTH  R0, R1      ; R0 = 0xFFFF8000 (符号扩展)
UXTH  R0, R1      ; R0 = 0x00008000 (无符号扩展)

; 字节序转换 (Endianness Conversion)
; R1 = 0x12345678 (大端存储)
REV   R0, R1      ; R0 = 0x78563412 (小端存储)
; R1 = 0x12345678
REV16 R0, R1      ; R0 = 0x34127856 (每半字内字节反转)
; R1 = 0x0000FF80 (大端16位有符号数 = -128)
REVSH R0, R1      ; R0 = 0xFFFFF80 (转小端+有符号扩展)
```

5.6.7 位域处理指令

Cortex-M3引入的位域处理指令极大地简化了寄存器位域操作——传统上需要多条移位+与或指令的组合才能完成的操作，现在只需一条指令。

指令格式	功能说明	示例
BFC Rd, #lsb, #width	位域清零	BFC R0, #4, #8 ; R0[11:4]=0
BFI Rd, Rn, #lsb, #width	位域插入	BFI R0, R1, #8, #4 ; R0[11:8]=R1[3:0]
CLZ Rd, Rm	计算前导零个数	CLZ R0, R1 ; R1=0x00100000→R0=11
RBIT Rd, Rm	位反转 (bit0↔bit31, ...)	RBIT R0, R1
UBFX Rd, Rn, #lsb, #width	无符号位域提取	UBFX R0, R1, #4, #8
SBFX Rd, Rn, #lsb, #width	有符号位域提取	SBFX R0, R1, #4, #8

```
; 位域操作示例
; BFC — 清除R0的bit[7:4]为0
BFC    R0, #4, #4    ; R0[7:4] = 0000

; BFI — 将R1的低8位插入到R0的bit[15:8]
BFI    R0, R1, #8, #8 ; R0[15:8] = R1[7:0]

; UBFX — 从R1提取bit[11:4] (无符号)
UBFX    R0, R1, #4, #8 ; R0 = R1[11:4], 高位补0

; SBFX — 从R1提取bit[11:4] (有符号)
SBFX    R0, R1, #4, #8 ; R0 = R1[11:4], 符号扩展至32位

; CLZ — Count Leading Zeros
MOV     R0, #0x00100000
CLZ     R1, R0        ; R1 = 11 (有11个前导零)

; RBIT — 完全位反转
MOV     R0, #0x80000000 ; bit31=1
RBIT    R1, R0        ; R1 = 0x00000001 (bit0=1)
```

5.6.8 比较和测试指令

比较和测试指令用于更新APSR标志位，通常后跟条件分支或IT指令块。它们总是更新APSR，无需S后缀。

指令格式	功能说明	示例
CMP Rn, <op2>	比较 (Rn - op2)，更新N/Z/C/V，丢弃结果	CMP R0, #0
CMN Rn, <op2>	比较取负 (Rn + op2)，更新N/Z/C/V	CMN R0, #1 ; 检查R0==-1
TST Rn, <op2>	位测试 (Rn AND op2)，更新N/Z/C	TST R0, #0x80 ; 测试bit7
TEQ Rn, <op2>	相等测试 (Rn EOR op2)，更新N/Z/C	TEQ R0, R1 ; 测试相等

```
; 比较和测试常用模式
; CMP — 比较大小
CMP    R0, #100
BGT    greater_than ; 如果 R0 > 100 跳转（有符号）

; CMN — 检测R0 == -1
CMN    R0, #1      ; R0 + 1 == 0 → Z=1
BEQ    is_minus_one

; TST — 位掩码测试
TST    R0, #(1<<3) ; 测试bit 3
BNE    bit_is_set   ; 非零 = bit3=1

; TEQ — 相等性测试（不影响C标志）
TEQ    R0, R1       ; R0 XOR R1
BEQ    equal        ; XOR结果为零=相等
```

5.6.9 程序流控制指令

程序流控制指令是任何非平凡程序的骨架。Cortex-M3/M4提供了六种分支/控制机制，从简单的无条件跳转到强大的IT条件块，覆盖所有控制流场景。

1. 无条件分支 B / BX

```
; B — 直接分支，偏移可达±16MB（32位版本）
B    target    ; 无条件跳转到 target

; BX — 分支并可选切换状态（Cortex-M仅Thumb，等效于B）
BX   R0        ; 跳转到R0中的地址
```

2. 函数调用 BL / BLX

BL (Branch with Link) 将返回地址保存到LR (R14)，然后跳转。BLX同时可切换状态（在Cortex-M中通常无此需求）。如果函数内部还有调用（嵌套调用），必须在入口处PUSH LR，否则LR会被覆盖。

```
; BL — 函数调用
BL    my_function ; LR = 下一条指令地址，跳转到 my_function
; ...
my_function:
PUSH  {R4-R7, LR} ; 保存现场和LR（如果此函数还需调用其他函数）
; 函数体 ...
BL    another_func ; 嵌套调用（LR会被覆盖，需要先保存）
; ...
POP   {R4-R7, PC} ; 恢复现场并返回到调用者
```

3. 条件分支 B<cond>

14个条件码（见5.4.2节）可用于条件分支，根据APSR标志位决定是否跳转。条件分支仅16位编码时有±256字节范围（32位版本±1MB）。

```
; 条件分支示例：if-else
CMP   R0, #10    ; 比较 R0 和 10
BLT   less_than  ; R0 < 10 → less_than
BGT   greater_than ; R0 > 10 → greater_than
; R0 == 10:
MOV   R1, #0
B     end_if
less_than:
MOV   R1, #-1
B     end_if
greater_than:
MOV   R1, #1
end_if:
; R1 = -1 (R0<10), 0 (R0==10), 1 (R0>10)
```

4. 比较并分支 CBZ / CBNZ

CBZ (Compare and Branch if Zero) 和 CBNZ (Compare and Branch if Non-Zero)

将比较和分支合并为一条指令，不影响APSR。仅支持前向分支（0~126字节）。

```
; CBZ/CBNZ 示例: while循环
MOV    R0, #10
loop:
; 循环体 ...
SUBS   R0, R0, #1    ; 递减
CBNZ   R0, loop      ; 如果 R0 != 0, 继续循环
; R0 == 0, 退出循环
; 等效于 C 语言的 while(--counter > 0)
```

5. IF-THEN (IT) 指令块

IT指令是Thumb-2最创新的特性之一。它允许在单个IT块内最多条件执行4条后续指令，无需传统的条件分支跳转，大幅减少代码尺寸和流水线刷新。

```
; IT 指令格式
; IT<x><y><z>
; x/y/z = T (Then, 条件为真执行) 或 E (Else, 条件为假执行)
; 支持的格式: IT, ITT, ITE, ITTT, ITTE, ITET, ITEE,
;             ITTTT, ITTTE, ITTET, ITTEE, ITETT, ITETE, ITEET, ITEEE
;
; 示例1: IT块执行if-else
CMP    R0, #0
ITE    EQ          ; If-Then-Else (2条指令)
MOVEQ  R1, #100    ; 如果 R0==0: R1=100
MOVNE  R1, #200    ; 否则: R1=200

; 示例2: 4条指令的IT块
CMP    R0, #10
ITTTE  GT          ; If-Then-Then-Then-Else
MOVGT  R1, #1      ; R0>10: 执行
ADDGT  R2, R2, R1   ; R0>10: 执行
SUBGT  R3, R3, #1   ; R0>10: 执行
MOVLE  R1, #0       ; R0<=10: 执行
```

注意：关键规则：IT块内的16位指令不更新APSR标志位！这是ARMv7-M的明确规定，以避免后续条件码被破坏。如果需要更新APSR，可以使用32位带S后缀的指令。

6. 表分支 TBB / TBH

TBB (Table Branch Byte) 和 TBH (Table Branch Halfword)

用于实现switch-case语句的高效跳转表。两种指令都仅支持前向分支。

```
; TBB — 字节偏移跳转表 (每项1字节, 偏移范围0~510字节)
TBB    [PC, R0]    ; PC = PC + 2*Table[PC + R0]
```

```

; branch_table:
; DCB (case0 - branch_table)/2
; DCB (case1 - branch_table)/2
; ...

; TBH — 半字偏移跳转表 (每项2字节, 偏移范围0~131070字节)
TBH    [PC, R0, LSL #1]; PC = PC + 2*Table[PC + R0*2]
; branch_table:
; DCI (case0 - branch_table)/2
; DCI (case1 - branch_table)/2
; ...

```


5.6.10 饱和运算指令

饱和运算（Saturation）将超出目标位宽的值「箝位」到最大值/最小值，而非截断（产生环绕错误）。这避免了信号处理中因溢出截断导致的波形畸变——如音频信号从最大值突然跳变到最小值。

指令格式	功能说明	示例
SSAT Rd, #bit_pos, Rn{,shift}	有符号饱和到指定位宽	SSAT R0, #8, R1 ; 饱和到8位[-128,127]
USAT Rd, #bit_pos, Rn{,shift}	无符号饱和到指定位宽	USAT R0, #12, R1 ; 饱和到12位[0,4095]

```
; 饱和运算示例
; SSAT — 有符号饱和
MOV    R1, #200      ; 200 超出有符号8位范围 [-128, 127]
SSAT   R0, #8, R1     ; R0 = 127 (饱和到最大值)
; 此时 Q 标志 = 1

; USAT — 无符号饱和
MOV    R1, #5000      ; 5000 超出12位无符号范围 [0, 4095]
USAT   R0, #12, R1    ; R0 = 4095 (饱和到最大值)

; 带移位的饱和（常用于定点运算）
SSAT   R0, #16, R1, LSL #2 ; 将(R1<<2)饱和到16位有符号范围
; Q标志: 发生饱和时置1，需软件清除（写APSR.Q=0）
```

提示：饱和运算广泛应用于：音频PCM采样（16位→8位压缩）、图像像素处理（避免溢出产生伪影）、电机控制（PID输出限幅）和通信信号处理。

5.6.11 异常相关指令

指令格式	功能说明	示例
SVC #imm	Supervisor Call, 触发SVC异常 (系统调用)	SVC #0x01 ; 调用系统服务
BKPT #imm	Breakpoint, 触发调试断点异常	BKPT #0 ; 软件断点
CPSID i	关中断 (设置PRIMASK=1)	CPSID i
CPSID f	关故障异常 (设置FAULTMASK=1)	CPSID f
CPSIE i	开中断 (清除PRIMASK=0)	CPSIE i
CPSIE f	开故障异常 (清除FAULTMASK=0)	CPSIE f

```
; SVC — 系统调用 (RTOS内核API入口)
SVC    #0          ; 触发SVC异常, 进入SVC_Handler

; BKPT — 调试断点
BKPT   #0          ; 进入DebugMonitor或HardFault

; 临界区保护 (关中断/开中断)
CPSID  i           ; 关中断 → 进入临界区
; ... 临界区代码 (不可被中断打断) ...
CPSIE  i           ; 开中断 → 退出临界区
```

警告： CPSID/CPSIE影响整个处理器的中断响应。在RTOS中， 应使用OS提供的临界区API（如taskENTER_CRITICAL()）， 而非直接操作CPS。

5.6.12 休眠模式指令

指令格式	功能说明	示例
WFI	Wait For Interrupt, 等待中断唤醒	WFI ; 进入休眠, 中断唤醒后继续
WFE	Wait For Event, 等待事件唤醒	WFE ; 进入休眠, 事件/中断唤醒
SEV	Send Event, 发送事件 (唤醒其他核)	SEV ; 多核系统中唤醒等待核

- WFI vs WFE: WFI由中断 (包括被PRIMASK屏蔽的) 唤醒; WFE可由外部事件 (RXEV信号)、其他核的SEV指令或中断唤醒。WFE更灵活但需要硬件事件信号支持。
- 典型应用: 在RTOS空闲任务中执行WFI/WFE以降低功耗; 在多核系统中用WFE+SEV实现核间同步。

```
; RTOS 空闲任务典型模式
idle_task:
WFI          ; 等待任意中断
B    idle_task ; 被唤醒后检查任务就绪列表
```

5.6.13 存储器屏障指令

存储器屏障是现代处理器系统级编程中最重要的概念之一。由于流水线、写缓冲和总线系统的并行性，指令的执行顺序和存储器访问的实际完成顺序可能不同。屏障指令强制顺序约束。

指令	全称	功能说明	典型使用场景
DMB	Data Memory Barrier	确保DMB之前的所有数据存储器访问完成于之后的访问之前	多核共享内存同步
DSB	Data Synchronization Barrier	确保DSB之前的所有存储器访问完成，才执行后续指令	更新系统控制寄存器后
ISB	Instruction Synchronization Barrier	清空流水线，确保ISB之后的指令从更新后的上下文获取	修改CPSR/CONTROL后

屏障链示例

```
; 关键示例：使能FPU的正确顺序
; 在Cortex-M4上使能FPU必须先设置CPACR，再执行屏障
LDR    R0, =0xE000ED88      ; SCB->CPACR 地址
LDR    R1, [R0]
ORR     R1, R1, #(0xF << 20) ; 设置CP10和CP11为完全访问
STR     R1, [R0]             ; 写入CPACR
DSB                                ; 确保CPACR写入完成
ISB                                ; 清空流水线，后续可安全使用FPU指令
```

多核共享内存同步

```
; 多核共享内存同步
; Core0写入共享数据
STR     R0, [shared_data]
DMB                                ; 确保写入对Core1可见
STR     R1, [flag]              ; 然后设置标志

; Core1读取
LDR     R0, [flag]
CMP     R0, #1
BNE     wait

DMB                                ; 确保看到标志后，共享数据也可见
LDR     R1, [shared_data]       ; 安全读取共享数据

注意：忽略存储器屏障是多核/系统级编程中最常见的bug来源之一。规则：
写共享数据→DMB→写标志；读标志→DMB→读共享数据。
```

5.6.14 ~ 5.6.15 其他指令与不支持的指令

其他指令

指令	功能说明	示例
NOP	空操作（什么都不做，可用作延时填充）	NOP
CLREX	清除本地独占监视器（强制后续STREX失败）	CLREX
PLD	预加载数据到缓存（Cortex-M3/M4无缓存，执行NOP）	PLD [R0]
PLI	预加载指令到缓存（Cortex-M3/M4执行NOP）	PLI [PC, #100]
YIELD	多线程yield提示（Cortex-M3/M4执行NOP）	YIELD

不支持的指令

- Cortex-M不支持32位ARM指令（如ADDEQ、LDMFD等传统ARM语法）——所有指令必须使用Thumb/Thumb-2编码。
- Cortex-M3不支持DSP/SIMD指令（见5.7节Cortex-M4特有指令）和FPU指令。
- Cortex-M不支持协处理器指令（MRC/MCR等），所有外设通过内存映射方式访问。
- PLD/PLI/YIELD虽然汇编器接受，但在无缓存的Cortex-M3/M4上实际执行NOP——它们是为了与更高级处理器（M7）保持代码兼容性。

5.7 Cortex-M4 特有指令 (DSP / SIMD / FPU)

Cortex-M4在M3的基础上增加了DSP扩展、SIMD并行处理指令和可选的单精度浮点单元（FPU）。这些扩展使其成为数字信号控制（DSC）领域的理想选择。

DSP扩展概述

- 增强的乘法累加（MAC）指令：支持多路并行MAC操作
- SIMD（单指令多数据）：在一个32位寄存器中同时操作4个8位或2个16位数据
- 饱和算术：自动处理溢出，保持数据在有效范围内
- 可选单精度浮点（FPv4-SP）：兼容IEEE 754标准

SIMD 数据类型

SIMD指令将一个32位寄存器视为包含多个小数据元素的向量：4×8位或2×16位。所有SIMD操作同时对多个元素执行相同的运算，大幅提升处理吞吐量。

```
; SIMD 数据布局 (一个32位寄存器)
; 4 x 8-bit: [byte3] [byte2] [byte1] [byte0]
; 2 x 16-bit: [ halfword1 ] [ halfword0 ]
```

SIMD 指令前缀规则

前缀	全称	含义	示例
S	Signed	有符号运算（环绕溢出）	SADD8, SSUB16
Q	Signed + Saturating	有符号饱和运算	QADD8, QSUB16
SH	Signed + Halving	有符号减半运算（防溢出）	SHADD8, SHSUB16
U	Unsigned	无符号运算（环绕溢出）	UADD8, USUB16
UQ	Unsigned + Saturating	无符号饱和运算	UQADD8, UQSUB16
UH	Unsigned + Halving	无符号减半运算（防溢出）	UHADD8, UHSUB16

常用SIMD操作

指令	功能	示例
SADD8 / UADD8	4路8位加法	SADD8 R0, R1, R2
SSUB8 / USUB8	4路8位减法	SSUB8 R0, R1, R2
SADD16 / UADD16	2路16位加法	SADD16 R0, R1, R2
SSUB16 / USUB16	2路16位减法	SSUB16 R0, R1, R2
SASX / UASX	高低半字交叉加减 (16位)	SASX R0, R1, R2
SSAX / USAX	高低半字交叉减加 (16位)	SSAX R0, R1, R2
QADD8 / QSUB8	4路8位饱和和加减	QADD8 R0, R1, R2

指令	功能	示例
QADD16 / QSUB16	2路16位饱和加减	QADD16 R0, R1, R2

增强MAC指令

指令	功能	应用场景
SMLABB Rd,Rn,Rm,Ra	有符号 $Rx[15:0] \times Ry[15:0] + Ra$	半字低位乘法累加
SMLABT Rd,Rn,Rm,Ra	有符号 $Rx[15:0] \times Ry[31:16] + Ra$	交叉半字乘法累加
SMLAD Rd,Rn,Rm,Ra	有符号双16位乘加 $(RxH \times RyH + RxL \times RyL) + Ra$	双路MAC
SMLSd Rd,Rn,Rm,Ra	有符号双16位乘减 $(RxH \times RyH - RxL \times RyL) + Ra$	双路乘减
SMLALD RdLo,RdHi,Rn,Rm	有符号双16位乘加→64位累加	长型双路MAC
SMUAD Rd,Rn,Rm	有符号双16位乘加 $RxH \times RyH + RxL \times RyL$	双路乘法求和
SMUSD Rd,Rn,Rm	有符号双16位乘减 $RxH \times RyH - RxL \times RyL$	双路乘法求差

打包/解包指令

指令	功能	示例
PKHBT Rd,Rn,Rm{,LSL #n}	打包半字（取 $Rn[15:0]$ 和 $Rm[31:16]$ ）	PKHBT R0, R1, R2
PKHTB Rd,Rn,Rm{,ASR #n}	打包半字（取 $Rn[31:16]$ 和 $Rm[15:0]$ ）	PKHTB R0, R1, R2
SXTA R0,R1,R2{,ROR #n}	有符号扩展+加法	扩展后与目标相累加
UXTA R0,R1,R2{,ROR #n}	无符号扩展+加法	扩展后与目标相累加

浮点指令（V-前缀）

Cortex-M4可选FPU（FPv4-SP）提供单精度浮点运算。所有浮点指令以V为前缀，使用寄存器组S0~S31（32个单精度寄存器，可配对为D0~D15双精度寄存器）。

指令格式	功能	示例
VADD.F32 Sd, Sn, Sm	浮点加法	VADD.F32 S0, S1, S2
VSUB.F32 Sd, Sn, Sm	浮点减法	VSUB.F32 S0, S1, S2
VMUL.F32 Sd, Sn, Sm	浮点乘法	VMUL.F32 S0, S1, S2
VDIV.F32 Sd, Sn, Sm	浮点除法	VDIV.F32 S0, S1, S2
VMLA.F32 Sd, Sn, Sm	浮点乘加 $Sd += Sn \times Sm$	VMLA.F32 S0, S1, S2
VMLS.F32 Sd, Sn, Sm	浮点乘减 $Sd -= Sn \times Sm$	VMLS.F32 S0, S1, S2
VNMUL.F32 Sd, Sn, Sm	浮点乘后取反 $Sd = -(Sn \times Sm)$	VNMUL.F32 S0, S1, S2
VNMLA.F32 Sd, Sn, Sm	负乘加 $Sd = -(Sn \times Sm + Sa)$	融合MAC的反
VABS.F32 Sd, Sm	浮点绝对值	VABS.F32 S0, S1
VNEG.F32 Sd, Sm	浮点取反	VNEG.F32 S0, S1
VSQRT.F32 Sd, Sm	浮点平方根	VSQRT.F32 S0, S1
VCMP.F32 Sd, Sm	浮点比较（更新FPSCR）	VCMP.F32 S0, S1
VCVT.F32.S32 Sd, Sm	整数→浮点转换	VCVT.F32.S32 S0, S1
VCVT.S32.F32 Sd, Sm	浮点→整数转换	VCVT.S32.F32 S0, S1

指令格式	功能	示例
VLDR Sd, [Rn, #off]	浮点加载（内存→S寄存器）	VLDR S0, [R1, #4]
VSTR Sd, [Rn, #off]	浮点存储（S寄存器→内存）	VSTR S0, [R1, #4]
VLDM Rn!, {S list}	浮点多寄存器加载	VLDM R0!, {S0-S3}
VSTM Rn!, {S list}	浮点多寄存器存储	VSTM R0!, {S0-S3}
VPOP {S list}	浮点弹出（从堆栈）	VPOP {S0-S3}
VPUSH {S list}	浮点压入（到堆栈）	VPUSH {S0-S3}

FPU 使能步骤

在Cortex-M4上使用FPU之前，必须通过SCB->CPACR寄存器使能CP10和CP11协处理器访问权限。否则任何FPU指令将导致UsageFault（NOCP — No Coprocessor）异常。

```
; — Cortex-M4 FPU 使能代码 —
; 方法1: 汇编
LDR    R0, =0xE000ED88      ; SCB->CPACR 地址
LDR    R1, [R0]
ORR     R1, R1, #(0xF << 20) ; CP10=0b11, CP11=0b11 (完全访问)
STR     R1, [R0]
DSB                                ; 数据同步屏障
```



```
ISB          ;指令同步屏障
```

```
;现在可以安全使用FPU指令
```

```
;方法2: C语言 (CMSIS-Core)
```

```
SCB->CPACR |= ((3UL << 10*2) | (3UL << 11*2)); // 使能CP10和CP11
```

警告：在RTOS中，FPU上下文的保存和恢复需要特殊处理（S0-S31和FPSCR共132字节）。大多数RTOS（FreeRTOS、RT-Thread等）提供了FPU任务切换支持，需启用相应配置宏。

5.8 桶形移位器 (Barrel Shifter)

桶形移位器是ARM处理器的一项经典硬件特性。它允许在数据处理指令的第二个操作数上执行移位操作——不需要额外的移位指令，移位和运算在同一个周期内完成。

支持的移位类型

移位操作	说明	示例
ASR #n / Rs	算术右移	ADD R0, R1, R2, ASR #4
LSL #n / Rs	逻辑左移	ADD R0, R1, R2, LSL #2
LSR #n / Rs	逻辑右移	SUB R0, R1, R2, LSR #8
ROR #n / Rs	循环右移	AND R0, R1, R2, ROR #16
RRX	带进位循环右移1位 (C参与)	ADC R0, R1, R2, RRX

指令格式

```
; 格式: mnemonic Rd, Rn, Rm, {shift}
ADD    R0, R1, R2, LSL #3 ; R0 = R1 + (R2 << 3)
SUB    R0, R1, R2, LSR #2 ; R0 = R1 - (R2 >> 2) [无符号右移]
MOV    R0, R1, LSL #8    ; R0 = R1 << 8
AND    R0, R1, R2, ROR #16 ; R0 = R1 & (R2 ROR 16)
```

支持桶形移位器的指令

- 数据处理指令：ADD, ADC, SUB, SBC, RSB, AND, ORR, EOR, BIC, ORN
- 比较测试指令：CMP, CMN, TST, TEQ
- 传送指令：MOV, MVN（仅一个操作数，但有移位变体）

存储器访问中的桶形移位器

桶形移位器也可以用于存储器访问指令的寄存器偏移寻址中，用于计算数组索引 (base + index × size)：

```
; 使用桶形移位器进行数组索引缩放
; uint32_t array[100];
; R1 = 基址, R2 = 索引 i
LDR    R0, [R1, R2, LSL #2] ; R0 = array[i] (每个元素4字节)

; uint16_t array[100];
LDRH   R0, [R1, R2, LSL #1] ; R0 = array[i] (每个元素2字节)

; 数据处理组合: 基址不变 + 按元素大小缩放偏移
ADD    R3, R1, R2, LSL #2 ; R3 = &array[i] (基址+i*4)
LDR    R0, [R3]           ; 等效于上面的一步LDR
```

提示：桶形移位器的优势是零开销：移位运算不需要额外的指令周期。合理利用可以显著减少指令数量和提升循环体内的数据处理效率。

5.9 访问特殊寄存器和特殊指令

访问特殊功能寄存器（PRIMASK、FAULTMASK、CONTROL等）和特殊指令（WFI、WFE等）有多种编程方法。不同方法在可移植性、效率和可读性方面各有权衡。

访问方法比较

方法	可移植性	推荐度	示例
CMSIS-Core 内联函数	高（跨ARM编译器）	★★★★★	<code>__WFI(); __enable_irq();</code>
编译器内建函数 (builtin)	低（编译器特定）	★★★★☆	<code>__builtin_arm_wfi();</code> (GCC)
内联汇编 (inline asm)	低	★★★☆☆	<code>__asm volatile("wfi");</code>
嵌入式汇编 (.s文件)	低	★★★☆☆	外部汇编源文件
编译器关键字/扩展	低	★★☆☆☆	<code>__irq</code> 关键字 (ARM Compiler)

CMSIS-Core 常用内联函数

函数	返回类型	功能
<code>__WFI()</code>	void	等待中断 (Wait For Interrupt)
<code>__WFE()</code>	void	等待事件 (Wait For Event)
<code>__SEV()</code>	void	发送事件 (Send Event)
<code>__ISB()</code>	void	指令同步屏障
<code>__DSB()</code>	void	数据同步屏障
<code>__DMB()</code>	void	数据存储器屏障
<code>__NOP()</code>	void	空操作
<code>__enable_irq()</code>	void	清除PRIMASK = 0 (开中断)
<code>__disable_irq()</code>	void	设置PRIMASK = 1 (关中断)
<code>__CLREX()</code>	void	清除独占监视器
<code>__RBIT(x)</code>	uint32_t	位反转
<code>__REV(x)</code>	uint32_t	字节反转
<code>__REV16(x)</code>	uint32_t	半字内字节反转
<code>__SSAT(val, bitpos)</code>	int32_t	有符号饱和
<code>__USAT(val, bitpos)</code>	uint32_t	无符号饱和
<code>__get_PRIMASK()</code>	uint32_t	读取PRIMASK
<code>__set_PRIMASK(x)</code>	void	设置PRIMASK
<code>__get_CONTROL()</code>	uint32_t	读取CONTROL
<code>__set_CONTROL(x)</code>	void	设置CONTROL
<code>__get_MSP()</code>	uint32_t	读取主堆栈指针
<code>__set_MSP(x)</code>	void	设置主堆栈指针

函数	返回类型	功能
<code>__get_PSP()</code>	<code>uint32_t</code>	读取线程堆栈指针
<code>__set_PSP(x)</code>	<code>void</code>	设置线程堆栈指针

提示：最佳实践：在应用代码中优先使用CMSIS-Core函数，以获得最佳的跨编译器和跨处理器可移植性。CMSIS-DSP库也免费提供，包含了针对Cortex-M4 SIMD和FPU深度优化的DSP算法实现。

```
// C语言示例：使用CMSIS-Core函数
#include "cmsis_compiler.h" // 或 CMSIS 5 的 cmsis_gcc.h

void low_power_example(void) {
    __WFI();           // 进入低功耗模式，等待中断
}

void critical_section_example(void) {
    uint32_t primask = __get_PRIMASK();
    __disable_irq();   // 进入临界区
    // ... 临界区代码 ...
    __set_PRIMASK(primask); // 恢复之前的中断状态
}

uint32_t reverse_bits(uint32_t val) {
    return __RBIT(val); // 硬件位反转
}
```

学习总结与重点回顾

1. ISA演进历程

ARM ISA从ARMv4的纯32位ARM指令，经过ARMv4T引入16位Thumb提升代码密度，最终在ARMv7-M/v7E-M发展到Thumb-2技术。Cortex-M系列仅支持Thumb状态，不再需要ARM/Thumb模式切换。理解这一演进是掌握指令集设计思想的基础。

2. 向上兼容性

M0→M3→M4形成清晰的向上兼容路径。M0的Thumb-1代码可直接运行在M3/M4上；M3代码可在M4上运行。反向不兼容——使用高级特性（DSP/FPU/SIMD）的代码不能向下移植。选型时需在成本、功耗和性能间权衡。

3. UAL统一汇编语言

UAL的三大核心变化：(1) S后缀显式化——不再自动更新APSR；(2) 推荐三操作数格式——源和目标分离；(3) .N/.W后缀精确控制编码宽度。移植旧Thumb代码时务必检查S后缀的使用。

4. APSR四标志

N(Negative)、Z(Zero)、C(Carry/Borrow)、V(oVerflow)四个标志位是所有条件执行的基础。条件分支（14个条件码）、IT指令块、带进位运算（ADC/SBC）都依赖APSR。理解每个标志的置位规则是写出正确汇编的前提。

5. 九种寻址模式

立即数偏移（数组存取）、PC相对（文字池）、寄存器偏移（索引缩放）、后变址（自动递增）、多寄存器LDM/STM（块传输）、PUSH/POP（堆栈）、SP相对（局部变量）、非特权访问（OS系统调用）、独占访问（互斥锁）。每种模式都有最佳使用场景。

6. IF-THEN指令块

IT块可在一组最多4条指令上实现条件执行，消除了传统分支的流水线刷新开销。关键规则：IT块内16位指令不更新APSR。适用于小型if-else和条件赋值，避免深层嵌套。

7. 桶形移位器

内置桶形移位器实现「移位+运算」的单周期融合——ADD R0, R1, R2, LSL #3等价于先移位再加法，但只需一条指令。数据处理和存储器访问指令均可利用此特性，零开销实现数组索引缩放和快速乘法。

8. M4 DSP与浮点

Cortex-M4的核心增值：SIMD（4×8位或2×16位并行处理）、增强MAC指令（双路乘法累加）、饱和运算保护、可选单精度FPU（FPv4-SP）。这些扩展使M4适用于电机控制、音频处理、传感器融合等需要实时信号处理的场景。

9. 存储器屏障

DMB（数据屏障）→确保数据访问顺序；DSB（数据同步屏障）→等待所有访问完成；ISB（指令同步屏障）→清空流水线。在修改系统寄存器（CPACR/CONTROL）、多核共享内存、外设初始化时必须使用正确的屏障链。忽略屏障是最常见的系统级bug。

10. CMSIS-Core优先

ARM提供的CMSIS-Core内联函数（__WFI, __enable_irq, __REV等）是访问特殊指令和寄存器的最佳方式——跨编译器、跨处理器、零性能开销。CMSIS-DSP库更提供了针对M4 SIMD/FPU深度优化的数字信号处理算法，强烈推荐优先使用。

—— 全文完 ——

参考资料：《ARM Cortex-M3与Cortex-M4权威指南（第3版）》Joseph Yiu 著